

FastAPI Dashboard

Type: Code_components Version: v8 Status: Published Generated: 2026-05-11 21:35:41

FastAPI Dashboard

The dashboard is a FastAPI web UI (port 9900) providing real-time visibility and manual controls for the IW AI Core orchestration platform. It is the primary human interface for managing projects, work items, batches, documentation, code understanding, and background jobs.

Purpose

The dashboard surfaces everything the daemon writes to PostgreSQL in a browsable, interactive UI. Agents run in git worktrees and report results via `iw` CLI commands; the dashboard reads that same state and renders it. No business logic lives in the dashboard routers — they only validate inputs and delegate to the `orch/` layer.

Key Components

Application Factory

`dashboard/app.py` — `create_app()` bootstraps the FastAPI application:

- **Lifespan:** On startup, `mark_orphaned_runs()` flips crashed TestRuns to `error`, then `verify_instance_identity()` confirms the DB fingerprint matches (CR-00014). Failures are suppressed in test/operator contexts but propagate in production.
- **Session cookie:** Sets an `iw_chat_session` HttpOnly cookie (readable by JS for localStorage scoping) if absent.
- **Alembic guard middleware:** Throttles DB-schema-behind warnings at most once per 10 seconds.
- **Schema-mismatch handler:** Catches SQLAlchemy `ProgrammingError` / `UndefinedColumn` and returns a clear 503 with remediation steps (F-00079).
- **Jinja2 templates:** Registered on `app.state.templates` with custom filters: `intcomma`, `timeago`, `fmt_ts_time`, `localdt`. The `is_db_stale` global is also attached.
- **Health endpoint:** `GET /health` — used by browser_verification steps.

Routers

All routers are registered in `app.py:277–308`. Business logic stays in `orch/`; routers are thin.

Core navigation & project selection

Router	Prefix	Purpose
<code>projects.py</code>	<code>/projects</code>	Project list, creation modal, nav fragment
<code>project_dashboard.py</code>	<code>/project/{id}</code>	Per-project home
<code>project_pages.py</code>	<code>/project/{id}</code>	Per-project queue and history pages

Router	Prefix	Purpose
<code>running.py</code>	<code>/running</code>	System-wide running items view
<code>system.py</code>	<code>/system</code>	Status, config, all-active
<code>healthz.py</code>	<code>/healthz</code>	DB identity probe, unauthenticated

Work items, batches, actions

Router	Prefix	Purpose
<code>items.py</code>	<code>/project/{id}/items/{item_id}</code>	Item detail tabs: overview, design-doc, reports, artifacts, logs, fix cycles, execution report, evidences
<code>batches.py</code>	<code>/project/{id}/batches</code>	Batch list, batch detail, batch diagram (PNG/.drawio)
<code>actions.py</code>	<code>/actions/*</code>	htmx endpoints: approve, cancel, pause, resume, restart, create batch, etc.
<code>search.py</code>	<code>/project/{id}/search</code>	Project-scoped FTS search
<code>sse.py</code>	<code>/api/stream</code>	GET <code>/api/stream/events</code> — SSE for live updates

Code view (backed by `orch/rag/`)

Router	Prefix	Purpose
<code>code_ui.py</code>	<code>/project/{id}/code</code>	Code browser page, index status, architecture map, index/reindex/regen-map/delete-index actions, SSE for index progress
<code>code.py</code>	<code>/api/projects/{id}/code</code>	Module listing, module detail, symbol explainer
<code>code_qa.py</code>	<code>/api/projects/{id}/code/qa</code>	SSE streaming RAG Q&A with citations

Docs view (backed by `orch/doc_service.py`, `orch/doc_sections.py`, `orch/doc_diff.py`)

Router	Prefix	Purpose
<code>docs.py</code>	<code>/project/{id}/docs</code>	Library, detail, HTML/PDF view, versions, diffs, stale detection, link validation, lint warnings, IDE tab, editorial guides, DocGenerationJob lifecycle
<code>docs_global.py</code>	<code>/docs</code>	Cross-project doc search

Research view

Router	Prefix	Purpose
<code>research.py</code>	<code>/project/{id}/research</code>	List, detail, HTML view, search (filters <code>doc_type=research</code>)

Frontend-triggered runs

Router	Prefix	Purpose
<code>tests.py</code>	<code>/project/{id}/tests</code>	Launch per category, live log, Allure results, kill run
<code>quality.py</code>	<code>/project/{id}/quality</code>	Static analysis / quality gates (same shape as tests)

Operations

Router	Prefix	Purpose
<code>jobs_ui.py</code>	<code>/project/{id}/jobs</code>	Unified job table via <code>orch/jobs/aggregator.py</code>
<code>worktrees.py</code>	<code>/system/worktrees</code>	Git status of active agent worktrees
<code>daemon_control.py</code>	<code>/system/daemon</code>	Daemon lifecycle from UI
<code>_run_helpers.py</code>	—	Shared helpers for tests/quality routers

Dependencies

`dashboard/dependencies.py` — `get_db()` yields a SQLAlchemy `Session` per request using `SessionLocal` from `orch.db.session` (same sync ORM as the daemon).

Static Assets

`dashboard/static/` — prebuilt Tailwind CSS (`styles.css`), JS, and other assets. Regenerated via `make css`.

Templates

Directory	Purpose
<code>templates/pages/</code>	Full-page templates (extend <code>base.html</code>)
<code>templates/components/</code>	Reusable Jinja2 macros/includes
<code>templates/fragments/</code>	htmx partial responses — do NOT extend <code>base.html</code>
<code>templates/pdf/</code>	PDF rendering templates for WeasyPrint

Data Flow

```
graph TD
    Browser[Browser (htmx / SSE)]
    Dashboard[FastAPI Dashboard :9900]
    OrchDB[(PostgreSQL :5433)]
    DocService[orch/doc_service.py]
    DocDiff[orch/doc_diff.py]
    DocSections[orch/doc_sections.py]
    DocJob[DocGenerationJob (DB table)]
    DocJobPoller[orch/daemon/doc_job_poller.py]
    Agent[AI Agent (iw-doc-generator)]

    Browser -->| "htmx POST / SSE stream" | Dashboard
    Dashboard -->| "SQLAlchemy sync" | OrchDB
    Dashboard --> DocService
```

config:

layout: elk

flowchart TB

Browser["Browser (htmx / SSE)"]

Dashboard["FastAPI Dashboard :9900"]

OrchDB["PostgreSQL :5433"]

DocService["orch/doc_service.py"]

DocDiff["orch/doc_diff.py"]

DocSections["orch/doc_sections.py"]

DocJob["DocGenerationJob (DB table)"]

DocJobPoller["orch/daemon/doc_job_poller.py"]

Agent["AI Agent (iw-doc-generator)"]

```
Browser -->| "htmx POST / SSE stream" | Dashboard
Dashboard -->| "SQLAlchemy sync" | OrchDB
Dashboard --> DocService
```

```
Dashboard --> DocDiff
Dashboard --> DocSections

DocJob --> DocJobPoller
DocJobPoller --> Agent
Agent -->|"iw doc-update"| DocService
Agent -->|"iw doc-job-done"| DocJob
```